

DevOps Engineer

Interview Preparation Guide

Prepared for: Sanket Ajay Chopade

AWS · Terraform · Kubernetes · Jenkins · GitHub Actions · Argo CD · Docker

Contents

1. Resume & Experience-Based Questions
2. AWS Questions
3. Terraform Questions
4. Jenkins Questions
5. Docker Questions
6. Kubernetes Questions
7. GitHub Actions & GitOps Questions
8. Monitoring & Observability Questions
9. Security & DevSecOps Questions
10. Linux, Bash & Python Questions
11. Project-Based Questions
12. Behavioral & Situational Questions

1. Resume & Experience-Based Questions

Tell me about yourself.

- I am Sanket Chopade, a B.Tech graduate from RTM Nagpur University with 9 months of DevOps internship experience at Hisan Labs, Pune.
- I supported a microservices-based e-commerce application on AWS as part of a 4-member DevOps team.
- My core competencies include AWS (EKS, IAM, VPC, S3, RDS), Terraform, Kubernetes, Docker, Jenkins, GitHub Actions, Argo CD, and monitoring with Prometheus and Grafana.
- I have hands-on project experience with a full Three-Tier DevSecOps deployment on EKS and a Go Cloud-Native GitOps delivery pipeline.
- I am passionate about infrastructure automation, CI/CD, and building secure, scalable cloud-native systems.

Walk me through your resume.

- I completed my B.Tech at RTM Nagpur University (2021–2025) and joined Hisan Labs as a DevOps Engineer Intern in September 2025.
- During the internship I designed and maintained Jenkins CI/CD pipelines for Terraform provisioning, frontend, and backend deployments.
- I developed Terraform modules for AWS resources (VPCs, Security Groups, S3, RDS, EKS) and managed remote state for collaborative deployments.
- I built and optimised Docker images and managed containerised workloads on EKS using Deployments, Services, Ingress, ConfigMaps, Secrets, and HPA.
- I integrated monitoring using Prometheus, Grafana, Datadog, and CloudWatch, and implemented security scanning via SonarQube and Trivy.
- Outside the internship I built two notable projects: a Three-Tier DevSecOps Project and a Go Cloud-Native GitOps Delivery Pipeline.

Why did you choose DevOps?

- I was drawn to DevOps because it sits at the intersection of software development and infrastructure — two areas I find equally exciting.
- I enjoy the problem-solving aspect of automating deployments, reducing toil, and enabling faster, more reliable software delivery.
- AWS and Kubernetes ecosystems gave me the opportunity to work with modern, industry-standard tooling from day one.

What were your daily responsibilities at Hisan Labs?

- Maintaining Jenkins pipelines and resolving pipeline failures for infrastructure and application deployments.
- Writing and reviewing Terraform code for AWS resource provisioning.
- Monitoring application health via Prometheus, Grafana, and Datadog dashboards and responding to alerts.
- Managing Kubernetes workloads — scaling deployments, debugging pods, and updating Helm releases.
- Reviewing code quality reports from SonarQube and coordinating vulnerability remediation with developers.
- Updating operational documentation and managing AWS IAM access for team members.

2. AWS Questions

What is IAM? Explain users, groups, roles, and policies.

- IAM (Identity and Access Management) controls who can access AWS resources and what actions they can perform.
- User: A permanent identity for a person or application with long-term credentials.
- Group: A collection of users that share the same permissions — simplifies bulk permission management.
- Role: A temporary identity assumed by AWS services (e.g., EC2, Lambda) or federated users. No long-term credentials.
- Policy: A JSON document defining allowed or denied actions on AWS resources. Attached to users, groups, or roles.
- Least privilege: Grant only the minimum permissions required to perform a task — critical security best practice.

Scenario: A developer requests AdministratorAccess. What would you do?

- Decline the request immediately — AdministratorAccess violates the principle of least privilege.
- Ask the developer what specific tasks they need to perform and identify the exact AWS services required.
- Create a custom IAM policy granting only those specific permissions (e.g., S3 read-write, EC2 describe).
- If temporary elevated access is required, assign the permissions via an IAM role with a time-bound session using STS AssumeRole.
- Log all actions with AWS CloudTrail for audit purposes.

Explain VPC architecture: subnets, NAT Gateway, Internet Gateway, Security Groups vs NACLs.

- VPC (Virtual Private Cloud): A logically isolated section of AWS where you launch resources in a defined IP range.
- Public Subnet: Has a route to the Internet Gateway — hosts load balancers, bastion hosts, NAT Gateways.
- Private Subnet: No direct internet route — hosts application servers, databases, and EKS worker nodes.
- Internet Gateway (IGW): Allows communication between VPC resources and the internet.
- NAT Gateway: Placed in a public subnet; allows private subnet instances to initiate outbound internet traffic (e.g., package downloads) without being directly reachable from the internet.
- Security Group: Stateful, instance-level firewall. Allows return traffic automatically.
- NACL: Stateless, subnet-level firewall. Rules for inbound and outbound must be defined separately.

Scenario: EC2 instance cannot access the internet. How do you troubleshoot?

- Check if the instance is in a public or private subnet.
- Public subnet: Verify an Internet Gateway is attached to the VPC and the subnet route table has a 0.0.0.0/0 → IGW route.
- Private subnet: Verify NAT Gateway exists in the public subnet and the route table has 0.0.0.0/0 → NAT Gateway.
- Check the Security Group inbound/outbound rules to ensure the required ports are open.
- Check NACLs — ensure both inbound and outbound allow the required traffic and ephemeral ports.
- Verify the instance has a public IP or Elastic IP if in a public subnet.
- Use VPC Flow Logs to identify dropped traffic.

Explain EKS architecture. Why EKS? Managed Node Group vs Self-Managed Nodes.

- EKS (Elastic Kubernetes Service) is a managed Kubernetes control plane hosted by AWS — users manage worker nodes.
- Control plane components (API server, etcd, scheduler, controller manager) are fully managed, highly available, and patched by AWS.
- Worker nodes run as EC2 instances (or Fargate pods) and register with the control plane using the node bootstrap script.
- Why EKS: Eliminates control-plane management overhead, integrates natively with AWS services (IAM, ALB, EBS, VPC CNI), and is SLA-backed.
- Managed Node Group: AWS provisions, patches, and terminates EC2 instances automatically; simplest option.
- Self-Managed Nodes: You control the AMI, instance lifecycle, and patching — useful for custom OS or GPU workloads.

Scenario: Pods are not scheduling in EKS. What would you check?

- Run `kubectl describe pod` and look at the Events section for scheduling errors.
- Check for Insufficient CPU/Memory: run `kubectl describe nodes` and inspect Allocatable vs Requests.
- Check node taints and pod tolerations — mismatched taints prevent scheduling.
- Check node selectors and affinity rules on the pod spec.
- Verify the Cluster Autoscaler is running and configured if nodes need to scale out.
- Confirm the node group IAM role has the correct permissions (e.g., `AmazonEKSWorkerNodePolicy`, `AmazonEC2ContainerRegistryReadOnly`).
- Check if the pod image can be pulled: `ImagePullBackOff` means ECR permissions or network issues.

ALB vs NLB? How does ALB Ingress Controller work?

- ALB (Application Load Balancer): Layer 7, supports HTTP/HTTPS, path-based and host-based routing, WebSockets, and gRPC. Ideal for web applications.
- NLB (Network Load Balancer): Layer 4, ultra-low latency, supports TCP/UDP/TLS. Ideal for non-HTTP traffic and real-time applications.
- AWS Load Balancer Controller runs as a pod in EKS and watches for Kubernetes Ingress resources.
- When an Ingress resource is created with the `alb` ingress class, the controller provisions an ALB in AWS with listener rules mapped to Kubernetes services.
- Target groups are created per service/pod, and health checks are managed automatically.

3. Terraform Questions

Explain the Terraform workflow and what a state file is.

- Workflow: Write (HCL code) → `terraform init` (download providers) → `terraform plan` (preview changes) → `terraform apply` (create/modify resources) → `terraform destroy` (teardown).
- State file (`terraform.tfstate`): A JSON file that maps Terraform resource definitions to real-world infrastructure.
- It tracks resource IDs, metadata, and dependencies, allowing Terraform to determine what changes are needed on the next apply.
- Without state, Terraform cannot know what it has already created — state is the source of truth.

Why use remote state? What is state locking?

- Remote state (e.g., S3 + DynamoDB): Stores the state file centrally so all team members use the same state — prevents drift and conflicts.
- State locking: When Terraform runs, it acquires a lock on the state (via DynamoDB) so no other engineer can run apply simultaneously.
- This prevents race conditions where two concurrent applies could corrupt the state or create duplicate resources.
- In our internship, we used an S3 backend with a DynamoDB lock table for all shared Terraform environments.

Why create Terraform modules? Explain module structure.

- Modules encapsulate reusable infrastructure patterns, enforce consistency, and reduce code duplication across environments.
- A module has: main.tf (resources), variables.tf (inputs), outputs.tf (exported values), and optionally versions.tf.
- Example: A vpc module accepts cidr_block, az_count as variables and outputs vpc_id, subnet_ids for use by other modules.
- We created modules for VPC, EKS, RDS, and S3 — each environment (dev/staging/prod) called the same modules with different variable values.

Scenario: Someone manually deletes a resource. What happens on the next Terraform apply?

- Terraform's state still records the resource as existing, but the real resource is gone.
- On terraform plan, Terraform detects the drift and shows a plan to re-create the deleted resource.
- On terraform apply, the resource is recreated to match the desired state in the configuration.
- To prevent accidental deletion, we use lifecycle { prevent_destroy = true } on critical resources like RDS and EKS clusters.
- terraform refresh can be used to sync the state with real infrastructure before running a plan.

4. Jenkins Questions

Explain your Jenkins pipeline stages.

- Stage 1 – Checkout: Clone the Git repository using GitHub webhooks triggered on push events.
- Stage 2 – Build: Compile the application (Maven for Java/Node for frontend).
- Stage 3 – Code Quality: Run SonarQube analysis and enforce Quality Gate pass/fail.
- Stage 4 – Security Scan: Run OWASP Dependency Check and Trivy image scan.
- Stage 5 – Docker Build & Push: Build Docker image, tag with build number, and push to ECR.
- Stage 6 – Deploy to EKS: Update the Kubernetes manifest image tag and apply via kubectl or trigger Argo CD sync.
- Stage 7 – Notification: Send Slack/email notification with build status.

How did Jenkins authenticate with AWS and store credentials?

- Jenkins stored AWS credentials (Access Key ID and Secret Access Key) in Jenkins Credentials Store (encrypted).
- Credentials were injected into pipeline stages using withCredentials or environment blocks — never hardcoded in Jenkinsfile.
- For Docker, Jenkins used a stored DockerHub/ECR credential binding.
- For Kubernetes, the kubeconfig file for the EKS cluster was stored as a Jenkins secret file credential.
- IAM roles can also be assigned to the Jenkins EC2 instance to avoid managing static credentials.

Scenario: Jenkins build succeeds but deployment fails. How do you troubleshoot?

- Check the Jenkins console output for the exact error in the deploy stage.
- Common cause 1 – kubectl auth failure: Verify the kubeconfig is valid and the IAM role/user has EKS access.
- Common cause 2 – Image pull error: Confirm the ECR image URI is correct and the EKS node role has `ecr:GetAuthorizationToken` permission.
- Common cause 3 – Kubernetes manifest error: Run `kubectl apply --dry-run=client -f manifest.yaml` to validate.
- Common cause 4 – Resource quota exceeded: Run `kubectl describe namespace` to check limits.
- Check pod status with `kubectl get pods` and `kubectl describe pod` to identify `CrashLoopBackOff` or `ImagePullBackOff`.

5. Docker Questions

Containers vs Virtual Machines — key differences.

- VMs: Include a full OS, hypervisor-managed, heavy (GBs), slower startup (minutes).
- Containers: Share the host OS kernel, isolated via namespaces/cgroups, lightweight (MBs), fast startup (seconds).
- Containers are immutable, portable, and ideal for microservices — the same image runs identically in dev, staging, and production.

Why use multi-stage Dockerfiles? Why Distroless images?

- Multi-stage builds: Use a builder stage (with SDK, compilers) to compile the application, then copy only the binary/artifact into a minimal final image.
- This removes build tools from the final image, reducing image size by 60–80% and the attack surface significantly.
- Distroless images: Contain only the application and its runtime dependencies — no shell, no package manager, no OS utilities.
- Distroless drastically reduces CVE exposure because there are fewer packages to have vulnerabilities.
- Example: `gcr.io/distroless/java` for Java apps, `gcr.io/distroless/base` for Go binaries.

Scenario: Docker image builds successfully but container crashes on startup. What do you do?

- Run `docker run` to see real-time logs. Check for missing environment variables, missing config files, or port conflicts.
- Run `docker logs` after the crash to review the exit message.
- Run `docker inspect` to check environment variables, volumes, and entrypoint.
- Test interactively: `docker run -it --entrypoint /bin/sh` to explore the container filesystem.
- Common causes: wrong ENTRYPOINT/CMD, missing ENV variables, dependency service not reachable, filesystem permission issues.

6. Kubernetes Questions

Explain Kubernetes architecture.

- Control Plane: API Server (entry point for all requests), etcd (distributed key-value store for cluster state), Scheduler (assigns pods to nodes), Controller Manager (ensures desired state).
- Worker Nodes: kubelet (communicates with API server, manages pods), kube-proxy (manages network rules for Services), Container Runtime (Docker/containerd).
- Pods are the smallest deployable unit — one or more containers sharing network and storage.
- In EKS, the control plane is managed by AWS; you manage worker node groups.

Deployment vs StatefulSet vs DaemonSet.

- Deployment: Manages stateless applications. Supports rolling updates, rollbacks. Pods are interchangeable (no stable identity).
- StatefulSet: Manages stateful applications (databases). Each pod gets a stable, unique identity (pod-0, pod-1) and persistent volume.
- In our Three-Tier project, MongoDB was deployed as a StatefulSet to preserve data identity across restarts.
- DaemonSet: Ensures a copy of a pod runs on every node. Used for log collectors (Fluentd), monitoring agents (Datadog), or network plugins.

Explain Kubernetes Ingress. Why use it?

- Ingress is a Kubernetes API object that manages external HTTP/HTTPS access to services inside the cluster.
- Without Ingress, each service would need its own LoadBalancer, creating one cloud load balancer per service — expensive and inefficient.
- Ingress consolidates routing: a single ALB handles all traffic and routes based on host headers or URL paths to different backend services.
- Example: /api → backend-service, / → frontend-service — all through one ALB.
- We used the AWS Load Balancer Controller as the Ingress controller in our EKS cluster.

Explain HPA (Horizontal Pod Autoscaler).

- HPA automatically scales the number of pod replicas based on observed CPU/memory utilisation or custom metrics.
- It queries the Kubernetes Metrics Server periodically and compares current utilisation against the target threshold.
- Example: If target CPU is 50% and current is 80%, HPA increases replicas to bring average CPU down.
- In our internship, we configured HPA with minReplicas: 2, maxReplicas: 10, and targetCPUUtilizationPercentage: 60.
- For scale-down, HPA waits for a cooldown period (default 5 minutes) to avoid thrashing.

Scenario: Application inaccessible but Pods are healthy. How do you troubleshoot?

- Verify pods are actually running: `kubectl get pods` — check STATUS is Running and READY is 1/1.
- Check the Service: `kubectl describe svc` — confirm selector matches pod labels and port mappings are correct.
- Check the Ingress: `kubectl describe ingress` — verify rules, host, and backend service.
- Check the ALB in AWS Console — confirm listener rules and target group health checks are passing.
- Test internal connectivity: `kubectl exec -it -- curl : to isolate DNS vs network issues.`
- Check Security Group rules on the ALB and worker nodes — ensure port 80/443 is open.
- Review pod logs for application-level errors: `kubectl logs` .

Scenario: Pod stuck in CrashLoopBackOff. What do you do?

- Run `kubectl logs --previous` to see logs from the last crashed container.
- Run `kubectl describe pod` to read Events — often reveals OOMKilled (out of memory) or startup probe failures.
- Common causes: application code error on startup, missing environment variable, misconfigured config file, missing secret, OOM.
- If OOMKilled: increase memory limits in the pod spec.
- If missing secret/configmap: verify the referenced resource exists in the same namespace.
- Run `kubectl get events --sort-by=.lastTimestamp -n` for a timeline of events.

7. GitHub Actions & GitOps Questions

Explain a GitHub Actions workflow structure.

- A workflow is a YAML file in `.github/workflows/` triggered by events (`push`, `pull_request`, `schedule`).
- It consists of jobs, each running on a runner (`ubuntu-latest`, `self-hosted`).
- Each job has steps — actions (uses: `actions/checkout@v3`) or shell commands (`run: docker build .`).
- Secrets are stored in GitHub repository settings and accessed via `${{ secrets.SECRET_NAME }}` — never exposed in logs.
- Jobs can depend on each other using `needs: [build]` to enforce sequential execution.

What is GitOps? Why use Argo CD?

- GitOps: A model where the Git repository is the single source of truth for infrastructure and application state. Changes are made via Git commits, and an agent continuously reconciles the cluster state with Git.
- Argo CD: A declarative GitOps tool for Kubernetes. It monitors Git repos for changes and automatically syncs the cluster.
- Why Argo CD: Provides a visual UI for deployment status, supports rollback to any Git commit, enforces drift detection, and integrates with RBAC.
- In our Go GitOps project, every push to the main branch triggered GitHub Actions to build/push the Docker image, update the Helm chart tag, and Argo CD automatically deployed the change.

Scenario: Code was merged but app wasn't updated in the cluster. What do you investigate?

- Check Argo CD UI — is the application in `Synced` or `OutOfSync` state?
- If `OutOfSync`: Verify the Argo CD application is pointing to the correct Git repository, branch, and path.
- Check if the GitHub Actions pipeline succeeded — confirm the image was built and the manifest/Helm values were updated in Git.
- If Auto-Sync is disabled, manually trigger a sync in Argo CD.
- Check Argo CD application events and pod logs: `kubectl logs -n argocd deployment/argocd-application-controller`.
- Verify Argo CD has permissions to access the Git repo (check SSH key or GitHub token configuration).

8. Monitoring & Observability Questions

Explain Prometheus architecture and exporters.

- Prometheus is a pull-based monitoring system. It scrapes metrics from HTTP endpoints (/metrics) at configured intervals.
- Components: Prometheus Server (scrapes, stores), Alertmanager (handles alerts), Pushgateway (for batch jobs), Exporters.
- Exporters: Agents that expose metrics in Prometheus format. Examples: node_exporter (host metrics), kube-state-metrics (Kubernetes object metrics), cAdvisor (container metrics).
- PromQL: Prometheus Query Language used to query and aggregate metrics. Example: `rate(http_requests_total[5m])` gives request rate over 5 minutes.
- In our project, we deployed Prometheus and Grafana using Helm charts (kube-prometheus-stack).

How do Grafana dashboards and alerts work?

- Grafana connects to Prometheus as a data source and visualises metrics using PromQL queries in panels.
- Dashboards are built from panels (graphs, gauges, stat, heatmaps) and can be imported from Grafana's community library.
- Alerts: Grafana evaluates a PromQL query on a schedule. If the result crosses a threshold, it sends notifications via configured channels (Slack, PagerDuty, email).
- We monitored CPU utilisation, memory usage, pod restart count, HTTP error rates, and request latency.

Scenario: CPU utilisation suddenly jumps to 95%. How do you investigate?

- Check Grafana dashboards immediately — identify which node and which pods are consuming the most CPU.
- Run `kubectl top pods -n` and `kubectl top nodes` to confirm.
- `kubectl describe pod` — check if HPA has already triggered a scale-out.
- Check application logs for unexpected load: `kubectl logs` — look for infinite loops, high-frequency polling, or GC pressure.
- Check CloudWatch metrics if running on EKS — EC2 CPU, network, and disk metrics may reveal the root cause.
- If a single pod is the outlier, restart it and trace back to the triggering event in the CI/CD history.

9. Security & DevSecOps Questions

What is SonarQube? What is a Quality Gate?

- SonarQube is a static code analysis platform that identifies bugs, code smells, security hotspots, and technical debt.
- Metrics analysed: code coverage, duplications, cyclomatic complexity, OWASP vulnerability patterns, and maintainability ratings.
- Quality Gate: A set of threshold conditions (e.g., coverage > 80%, no critical bugs) that a build must pass to proceed.
- In our Jenkins pipeline, a failed Quality Gate blocked the Docker build stage, preventing bad code from reaching production.

What is Trivy? How do you handle critical vulnerabilities?

- Trivy is an open-source vulnerability scanner for container images, filesystems, Git repos, and Kubernetes manifests.
- It scans OS packages and application dependencies against CVE databases (NVD, GitHub Advisories).
- In our pipeline, Trivy ran against every Docker image before deployment. Critical/High CVEs failed the build.
- Remediation steps: Update the base image to the latest patched version, upgrade vulnerable dependencies, or apply available patches.
- Trivy reports were stored as Jenkins build artifacts for audit trails.

How do you secure Kubernetes Secrets and Docker images?

- Kubernetes Secrets: Use external secret managers (AWS Secrets Manager + External Secrets Operator) instead of storing raw values in Git.
- Enable encryption at rest for etcd where secrets are stored.
- Apply RBAC to restrict which service accounts and namespaces can read specific secrets.
- Docker images: Use Distroless/minimal base images, run Trivy scans in CI, enable Docker Content Trust for image signing.
- Avoid running containers as root — use securityContext: runAsNonRoot: true in pod specs.
- Scan images regularly even after deployment using tools like Amazon ECR image scanning.

10. Linux, Bash & Python Questions

Key Linux commands for DevOps.

- grep: Search text patterns — grep -r 'ERROR' /var/log/ searches all logs for ERROR.
- awk: Process text fields — awk '{print \$1, \$3}' file.txt prints columns 1 and 3.
- sed: Stream editor for in-place substitution — sed -i 's/old/new/g' file.txt.
- find: Locate files — find /var/log -name '*.log' -mtime +7 finds logs older than 7 days.
- top/htop: Real-time process and CPU/memory monitoring.
- ps aux: List all running processes with user and memory details.
- netstat -tulnp / ss -tulnp: Show listening ports and associated processes.
- df -h: Disk usage; du -sh *: Directory-level disk usage.

Scenario: Disk at 100%. How do you identify the cause?

- Run df -h to identify which filesystem/mount is full.
- Run du -sh /* 2>/dev/null | sort -rh | head -20 to find the largest directories.
- Check /var/log — Docker, Jenkins, and application logs can grow unbounded.
- Run docker system prune to remove unused images, containers, and volumes.
- Check for large core dump files: find / -name 'core' -size +100M.
- If log files are the cause, implement log rotation using logrotate.

Write a Bash script to monitor disk usage and alert if above threshold.

- #!/bin/bash
- THRESHOLD=80
- USAGE=\$(df -h / | awk 'NR==2 {print \$5}' | tr -d '%')
- if [\$USAGE -gt \$THRESHOLD]; then
- echo "ALERT: Disk usage is at \${USAGE}% on \$(hostname)" | mail -s 'Disk Alert' admin@example.com
- fi
- This script extracts the root filesystem usage percentage, compares it to the threshold, and sends an email alert if exceeded.
- Can be scheduled via cron: */5 * * * * /opt/scripts/disk_monitor.sh

How would you automate AWS tasks using Python (Boto3)?

- Boto3 is the AWS SDK for Python — allows programmatic access to all AWS services.
- Example — list all running EC2 instances:
- import boto3; ec2 = boto3.client('ec2'); response = ec2.describe_instances(Filters=[{'Name': 'instance-state-name', 'Values': ['running']}])
- Common use cases: Start/stop EC2 instances on a schedule, copy S3 objects, rotate IAM access keys, trigger CodePipeline.
- Always use IAM roles (not hardcoded credentials) when running Boto3 scripts on EC2 or Lambda.

11. Project-Based Questions

Explain the Three-Tier DevSecOps Project architecture.

- Application: Full-stack MERN (MongoDB, Express, React, Node.js) deployed on Amazon EKS.
- Infrastructure: Provisioned entirely with Terraform — VPC, EKS cluster, EC2 (Jenkins), ECR, IAM roles.
- CI/CD: Jenkins pipeline (SonarQube → OWASP → Trivy → Docker Build/Push → Argo CD sync).
- Kubernetes resources: Frontend and backend as Deployments with ClusterIP Services; MongoDB as StatefulSet with PVC; ALB-based Ingress via AWS Load Balancer Controller.
- GitOps: Argo CD continuously monitored the Git repo and auto-synced changes to the cluster.
- Monitoring: Prometheus and Grafana deployed via Helm (kube-prometheus-stack) for cluster and application metrics.

Why did you choose each technology in the Three-Tier project?

- EKS: Managed Kubernetes — eliminates control-plane overhead while providing enterprise-grade orchestration.
- Terraform: Infrastructure as Code — repeatable, version-controlled, modular AWS resource provisioning.
- Jenkins: Mature CI tool with extensive plugin ecosystem for integrating SonarQube, Docker, Kubernetes, and AWS.
- Argo CD: GitOps model ensures the cluster state always matches Git — auditable, rollback-capable delivery.
- SonarQube + Trivy + OWASP: Shift-left security — catch code, dependency, and image vulnerabilities before production.
- Prometheus + Grafana: Industry-standard open-source monitoring — rich PromQL queries and customisable dashboards.

Explain the Go Cloud-Native GitOps Delivery Pipeline.

- A Go web application deployed on Amazon EKS using a fully automated GitOps pipeline.
- CI: GitHub Actions workflow triggered on push — ran unit tests, code quality checks, built a multi-stage Distroless Docker image, and pushed to ECR.
- CD: GitHub Actions updated the Helm chart image tag in Git; Argo CD detected the change and automatically synced the EKS cluster.
- Helm charts managed Deployment, Service, and Ingress resources declaratively.
- Distroless images ensured minimal attack surface — no shell or package manager in the final image.

Scenario: How would you scale the Three-Tier architecture for 1 million users?

- Frontend: Serve static assets via CloudFront (CDN) — offloads 90%+ of requests from the origin.
- Backend: Increase HPA maxReplicas, add Cluster Autoscaler to automatically provision additional EKS nodes.
- Database: Migrate MongoDB to a managed, horizontally scalable solution (Amazon DocumentDB or MongoDB Atlas) with read replicas.
- Load Balancer: ALB already scales automatically — ensure target group connection limits are sufficient.
- Caching: Introduce Amazon ElastiCache (Redis) for session data and frequently queried results.
- Observability: Add distributed tracing (AWS X-Ray or Jaeger) to identify bottlenecks under high load.

12. Behavioral & Situational Questions

Describe a difficult technical issue you solved during your internship.

- During the internship, one of our Jenkins pipelines was intermittently failing during the Terraform apply stage for EKS provisioning.
- The error was a race condition — the EKS cluster was created before the node groups were fully ready, causing subsequent kubectl commands to fail.
- I added explicit depends_on references in Terraform to enforce node group completion before any Kubernetes resource provisioning.
- I also added a health-check stage in Jenkins that polled the EKS API until all nodes reported Ready before proceeding.
- This resolved the intermittent failures entirely and improved overall pipeline reliability.

Situation: Production deployment fails 10 minutes before a major release. What do you do?

- Stay calm — communicate immediately with the team lead and stakeholders about the delay.
- Assess the failure: check Jenkins/Argo CD logs for the root cause in the first 2 minutes.
- If the fix is quick (config/env variable issue): fix, rerun the pipeline, and deploy.
- If the fix is complex: trigger a rollback via Argo CD (roll back to the last known-good Git commit) or kubectl rollout undo deployment/.
- Once stable, conduct a post-mortem: identify root cause, add pipeline checks to prevent recurrence.

Situation: Terraform apply accidentally destroys production resources. What is your recovery plan?

- Do not panic — check what was destroyed immediately via terraform state list and AWS Console.
- For stateless resources (Security Groups, subnets): re-run terraform apply — Terraform will recreate them.
- For stateful resources (RDS, EKS): restore from the most recent automated snapshot (RDS automated backups, EKS etcd backup).
- Notify the team and management transparently — document the incident timeline.
- Prevention: Add lifecycle { prevent_destroy = true } to all production-critical resources. Use Terraform Sentinel or OPA policies to block destroy in production workspaces. Require peer review on all terraform plan outputs before apply.

Why should we hire you for a DevOps Engineer role?

- I bring 9 months of real-world DevOps internship experience — not just theoretical knowledge — with production exposure to AWS, Kubernetes, Terraform, Jenkins, and GitOps.
- I have independently built and documented end-to-end DevSecOps pipelines incorporating security scanning, infrastructure as code, GitOps delivery, and full-stack monitoring.
- I have a strong troubleshooting mindset — I approach problems systematically, document resolutions, and implement preventive measures.
- I am a continuous learner — I stay updated with CNCF ecosystem developments and apply new tools pragmatically.
- I am collaborative, comfortable working in cross-functional teams, and take full ownership of the infrastructure I manage.

Prepared specifically for Sanket Ajay Chopade based on resume and internship experience at Hisan Labs. Best of luck with your interviews!